

Relatório de Auditoria de Segurança

ApexAlgo (CatDex)

Data da Auditoria: 09 de setembro de 2026

Escopo: Backend API + Frontend React + Deploy

Metodologia: Análise estática de código-fonte

Total de Achados: 12 vulnerabilidades identificadas

Stack Detectada:

- **Backend:** Python 3.11+ / FastAPI 0.135 / SQLAlchemy 2.0
- **Banco de Dados:** SQLite 3 (WAL mode)
- **ORM:** SQLAlchemy (sem Supabase, sem RLS)
- **Autenticação:** API Key global (MASTER_API_KEY) + session cookie HMAC
- **Frontend:** React 19 / Vite 8 / TailwindCSS 4
- **Deploy:** Docker Compose / nginx proxy / self-signed SSL
- **Criptografia:** Fernet (exchange API keys at rest)

1. Resumo Executivo

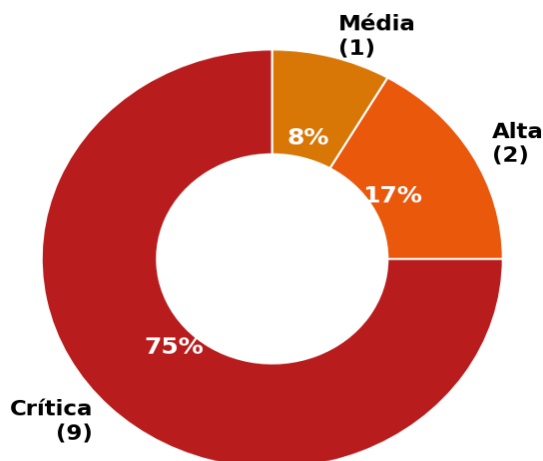
Esta auditoria identificou **12 vulnerabilidades de segurança** no projeto ApexAlgo, sendo **9 críticas**.

Principais Riscos:

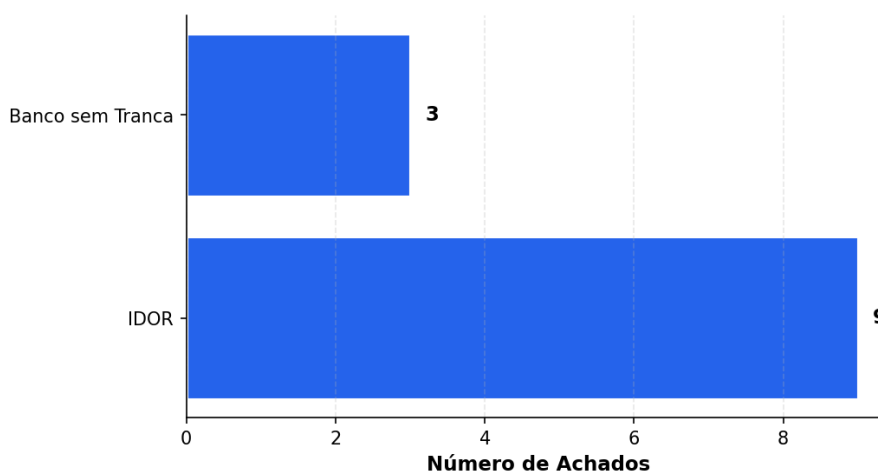
- **IDOR massivo:** 9 endpoints permitem acesso/modificação de recursos alheios
- **Ausência de isolamento:** Sistema single-user sem tenant_id nas tabelas
- **Manipulação de trades reais:** Atacante pode fechar posições live na exchange
- **Acesso a API keys:** Qualquer usuário pode ver/deletar/usar chaves alheias

Ponto crítico: O sistema foi projetado para self-hosting single-user, mas a arquitetura permite múltiplos bots e API keys sem isolamento. Qualquer usuário com a MASTER_API_KEY tem acesso total a todos os recursos.

Distribuição por Severidade



Achados por Categoria



2. Análise do Mecanismo de Isolamento

O projeto ApexAlgo NÃO possui mecanismo de isolamento multi-tenant. A aplicação é single-user por design: há apenas uma MASTER_API_KEY global que dá acesso total ao sistema. Não existem colunas user_id, tenant_id ou workspace_id em nenhuma tabela. Todas as queries buscam dados de todos os bots, positions, orders e exchange_keys sem filtro de ownership. O sistema foi projetado para uso pessoal (self-hosted), onde o único usuário é o dono da instância. Porém, a arquitetura permite criar múltiplos bots e múltiplas API keys, o que sugere que futuramente poderia ser multi-user - mas nesse caso, TODOS os endpoints precisariam ser refatorados para adicionar filtros de tenant.

3. Pontos Fortes

O sistema implementa várias boas práticas de segurança:

- ✓ Autenticação obrigatória: 43 de 47 endpoints protegidos com `verify_api_key`
- ✓ Criptografia de credenciais: API keys de exchanges criptografadas em repouso com Fernet
- ✓ Timing-safe comparison: HMAC usado para validar session cookies e API keys
- ✓ Rate limiting: Throttling de tentativas de autenticação falhadas (10 por minuto por IP)
- ✓ Secrets gerenciados: `.env` gerado automaticamente em primeiro boot, nunca commitado
- ✓ Permissões de arquivo: `.env` criado com `chmod 600`, não legível por outros usuários
- ✓ Histórico git limpo: Nenhum segredo encontrado commitado no histórico do repositório
- ✓ Sem XSS direto: Nenhum uso de `innerHTML`, `dangerouslySetInnerHTML` ou `eval` encontrado no frontend
- ✓ Sem hardcoded secrets: Nenhuma API key ou senha embutida no código-fonte
- ✓ SQLite WAL mode: Write-Ahead Logging evita locks de escrita/leitura

4. Achados Detalhados

4.1. IDOR

Severidade	CRÍTICA
Arquivo	backend/routers/trades.py:198
Descrição	DELETE /api/trades/positions/{position_id} não valida propriedade
Detalhe	Qualquer usuário autenticado pode deletar posições de qualquer bot sem verificar ownership. A rota busca P
Impacto	Perda de dados financeiros. Attacker pode apagar histórico de trades de outros bots.
Evidência	<pre>pos = db.query(Position).filter(Position.id == position_id).first() db.delete(pos)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/trades.py:401
Descrição	POST /api/trades/positions/{position_id}/close não valida propriedade
Detalhe	Qualquer usuário pode forçar fechamento de posição alheia (incluindo live orders na exchange real).
Impacto	Manipulação de trades reais. Attacker pode fechar posições live de outros usuários, causando perdas finan
Evidência	<pre>pos = db.query(Position).filter(Position.id == position_id).first() if not pos: return JSONResponse(status_code=404, ...)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/bots.py:236
Descrição	PUT /api/bots/{bot_id} permite editar bot alheio
Detalhe	Não há validação de ownership. Qualquer usuário autenticado pode modificar estratégias, API keys vinculad
Impacto	Takeover de bots. Attacker pode reconfigurar bots alheios para usar suas próprias exchange keys ou modifi
Evidência	<pre>bot = db.query(BotConfig).filter(BotConfig.id == bot_id).first() if not bot: raise HTTPException(status_code=404, ...)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/bots.py:332
Descrição	DELETE /api/bots/{bot_id} pode deletar bot alheio
Detalhe	Sem validação de ownership. Deleta bot e fecha todas posições abertas (incluindo live).

Impacto	Destruição de configurações e fechamento forçado de posições reais de outros usuários.
Evidência	<pre>bot = db.query(BotConfig).filter(BotConfig.id == bot_id).first() if not bot: raise HTTPException(status_code=404, ...)</pre>

Severidade	ALTA
Arquivo	backend/routers/keys.py:168
Descrição	GET /api/keys/{key_name}/balance expõe saldo de API key alheia
Detalhe	Qualquer usuário pode consultar saldo da carteira de qualquer API key cadastrada no sistema.
Impacto	Vazamento de informações financeiras sensíveis (saldos, holdings) de outros usuários.
Evidência	<pre>key_record = db.query(ExchangeKey).filter(ExchangeKey.name == key_name).first() if not key_record: raise HTTPException(status_code=404, ...)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/keys.py:233
Descrição	DELETE /api/keys/{key_name} pode deletar chave de API alheia
Detalhe	Sem validação de ownership. Attacker pode remover API keys de exchanges de outros usuários.
Impacto	Perda de acesso às exchanges. Bots param de funcionar, histórico de configuração perdido.
Evidência	<pre>key_record = db.query(ExchangeKey).filter(ExchangeKey.name == key_name).first() db.delete(key_record)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/keys.py:243
Descrição	POST /api/keys/{name}/swap executa swap em carteira alheia
Detalhe	Qualquer usuário pode executar market orders (swaps) usando API keys de outros usuários.
Impacto	Perdas financeiras diretas. Attacker pode drenar saldos executando swaps desfavoráveis ou movimentando
Evidência	<pre>key_record = db.query(ExchangeKey).filter(ExchangeKey.name == name).first() exchange = build_exchange_from_key(key_record) order = exchange.create_market_buy_order(...)</pre>

Severidade	ALTA
Arquivo	backend/routers/bots.py:520

Descrição	DELETE /api/bots/{bot_name}/cache limpa cache de bot alheio
Detalhe	Qualquer usuário pode limpar sinais, logs e trades simulados de qualquer bot via bot_name (string).
Impacto	Perda de dados históricos de backtest e sinais. Pode forçar re-execução de backtest custoso.
Evidência	<pre>bot = db.query(BotConfig).filter(BotConfig.name == bot_name).first() result = db.execute(text('DELETE FROM signals WHERE bot_name = :bn'), {'bn': bot_name})</pre>

Severidade	MÉDIA
Arquivo	backend/routers/trades.py:216
Descrição	POST /api/trades/positions/bulk-delete permite deleção em massa sem validação
Detalhe	Aceita lista de IDs e deleta todas posições sem verificar ownership de cada uma.
Impacto	Perda massiva de histórico de trades. Um único request pode apagar centenas de posições de múltiplos usu
Evidência	<pre>def bulk_delete_positions(ids: list[int] = Body(...), db: Session = Depends(get_db)): db.query(Position).filter(Position.id.in_(ids)).delete(...)</pre>

4.2. Banco sem Tranca

Severidade	CRÍTICA
Arquivo	backend/routers/trades.py:56
Descrição	GET /api/trades/positions não filtra por tenant/usuário
Detalhe	A aplicação não possui mecanismo de multi-tenancy. Todas as queries retornam dados de todos os usuários
Impacto	Não há isolamento de dados. Sistema single-user de fato, mas arquitetura permite criação de múltiplos bots
Evidência	<pre>query = db.query(Position.id, Position.exchange, Position.bot_name, ...) if symbol: query = query.filter(Position.symbol == formatted_symbol)</pre>

Severidade	CRÍTICA
Arquivo	backend/routers/bots.py:62
Descrição	GET /api/bots/ lista todos os bots sem filtro de ownership
Detalhe	Não existe coluna user_id ou tenant_id nas tabelas. Sistema é single-tenant de fato.
Impacto	Qualquer usuário vê todos os bots do sistema. Se fosse multi-user, seria vazamento total.

Evidência	<pre>def get_all_bots(db: Session = Depends(get_db)): return db.query(BotConfig).all()</pre>
------------------	--

Severidade	CRÍTICA
Arquivo	backend/routers/keys.py:131
Descrição	GET /api/keys lista todas API keys sem filtro de usuário
Detalhe	Todas as chaves de exchange cadastradas no sistema são visíveis para qualquer usuário autenticado.
Impacto	Vazamento de informações sensíveis: nomes de keys, exchanges usadas, status de conexão, bots vinculados
Evidência	<pre>def get_exchange_keys_status(db: Session = Depends(get_db)): keys = db.query(ExchangeKey).all()</pre>

5. Recomendações Priorizadas

P1 - CRÍTICO: Adicionar validação de ownership em TODOS os endpoints IDOR

Implementar user_id/tenant_id em todas as tabelas e filtrar todas as queries. Se o sistema permanecerá single-user, documentar isso explicitamente e adicionar warnings caso múltiplos usuários sejam adicionados futuramente.

P2 - CRÍTICO: Proteger endpoints de manipulação de exchange keys

DELETE /api/keys/{key_name} e POST /api/keys/{name}/swap devem validar ownership antes de executar. Swap de ativos é equivalente a transferência de dinheiro.

P3 - CRÍTICO: Validar ownership em fechamento de posições live

POST /api/trades/positions/{position_id}/close pode causar perdas financeiras reais. Adicionar verificação rigorosa de ownership + confirmação extra para live mode.

P4 - ALTA: Implementar RBAC ou sistema de permissões

Se o projeto evoluir para multi-user, implementar roles (admin, trader, viewer) e proteger endpoints administrativos.

P5 - MÉDIA: Adicionar auditoria de ações sensíveis

Logar todas operações de delete, swap, force-close com timestamp, IP e user identificador para forensics.

6. Issues para GitHub

As issues abaixo estão prontas para copiar e colar no GitHub:

ISSUE 1

Título: [Segurança] IDOR crítico em todos os endpoints de trades e bots

Labels: security, critical

Corpo da Issue:

Descrição

9 endpoints críticos não validam ownership, permitindo que qualquer usuário autenticado manipule recursos alheios.

Endpoints Afetados

- `DELETE /api/trades/positions/{position\_id}` - deletar posição alheia
- `POST /api/trades/positions/{position\_id}/close` - fechar posição live alheia
- `PUT /api/bots/{bot\_id}` - editar bot alheio
- `DELETE /api/bots/{bot\_id}` - deletar bot alheio
- `DELETE /api/bots/{bot\_name}/cache` - limpar cache alheio
- `GET /api/keys/{key\_name}/balance` - ver saldo alheio
- `DELETE /api/keys/{key\_name}` - deletar API key alheia
- `POST /api/keys/{name}/swap` - executar swap em carteira alheia
- `POST /api/trades/positions/bulk-delete` - deleção em massa sem validação

Impacto

****CRÍTICO:**** Perdas financeiras diretas, manipulação de trades reais, acesso a credenciais de exchanges.

Evidência

```
```python
backend/routers/trades.py:401
pos = db.query(Position).filter(Position.id == position_id).first()
Sem filtro de user_id/tenant_id
```
```

Correção Sugerida

1. Adicionar coluna `user\_id` ou `tenant\_id` em todas as tabelas (bots, positions, orders, exchange_keys)
2. Modificar TODAS as queries para filtrar por ownership:


```
```python
pos = db.query(Position).filter(
 Position.id == position_id,
 Position.user_id == current_user.id # ou tenant_id
).first()
```
```
3. Se o sistema permanecerá single-user, documentar explicitamente e adicionar validação em

startup que bloqueia multi-user

Critérios de Aceite

- [] Todas as queries filtram por ownership
- [] Testes unitários verificam rejeição de acesso a recursos alheios
- [] Documentação atualizada sobre modelo de autenticação (single vs multi-user)
- [] Se multi-user, migration criada para adicionar user_id/tenant_id



ISSUE 2

Título: [Segurança] Ausência de mecanismo de isolamento multi-tenant

Labels: security, critical, architecture

Corpo da Issue:

Descrição

O sistema não possui isolamento de dados. Todas as queries buscam recursos de todos os usuários sem filtro de tenant/ownership.

Problema Arquitetural

- Tabelas não possuem coluna `user\_id` ou `tenant\_id`
- Sistema é single-user por design (uma MASTER_API_KEY global)
- Porém, arquitetura permite múltiplos bots e API keys sem isolamento
- GET /api/bots/ retorna TODOS os bots do sistema
- GET /api/keys retorna TODAS as API keys cadastradas

Impacto

Se um segundo usuário for adicionado (ou se alguém roubar a MASTER_API_KEY), terá acesso total a todos os dados.

Decisão Necessária

O projeto deve escolher explicitamente:

Opção A - Single-User Enforcement:

- Documentar que é self-hosted single-user apenas
- Adicionar validação em startup que bloqueia criação de múltiplas sessões
- Renomear MASTER_API_KEY para OWNER_KEY
- Adicionar warning na UI se detectar múltiplos usuários

Opção B - Multi-Tenant:

- Adicionar user_id/tenant_id em todas as tabelas
- Implementar sistema de registro/authenticação por usuário
- Refatorar TODOS os endpoints para filtrar por tenant
- Adicionar RLS (Row Level Security) se migrar para PostgreSQL

Critérios de Aceite

- [] Decisão arquitetural documentada (A ou B)
- [] Se A: validações de single-user implementadas
- [] Se B: migration completa + refactor de queries + testes

[illegible]

ISSUE 3

Título: [Segurança] Swap endpoint permite manipulação financeira de carteiras alheias

Labels: security, critical, financial

Corpo da Issue:

Descrição

POST /api/keys/{name}/swap` executa market orders na exchange usando API key alheia, sem validar ownership.

Arquivo

```
`backend/routers/keys.py:243`
```

```
## Exploit
```

```
```bash
```

```
Atacante descobre nome de uma API key (via GET /api/keys)
```

```
curl -X POST https://api/victim.com/api/keys/victim_okx_key/swap \
```

```
-H "X-API-Key: $ATACANTE_MASTER_KEY" \
```

```
-d '{"from_asset": "BTC", "to_asset": "DOGE", "amount": 1000000, "amount_type": "from"}'
```

```
BTC da vítima é trocado por DOGE (perda financeira)
```

...

## ## Impacto

**\*\*CRÍTICO:\*\*** Drenagem de fundos. Atacante pode:

- Trocar ativos valiosos por memecoins ilíquidos
- Forçar swaps em momentos de baixa liquidez (slippage alto)
- Mover fundos para pares que não podem ser revertidos

## ## Correção

```
python
```

```
@router.post("/{name}/swap")
```

```
def execute quick swap(name: str, payload: dict, db: Session, current user =
```

Depends(get\_current\_user()):

```
key_record = db.query(ExchangeKey).filter(
```

```
ExchangeKey.name == name,
```

```
ExchangeKey.user_id == current_user.id # ADICIONAR
```

```
).first()
if not key_record:
 raise HTTPException(status_code=404, detail="Key not found or access denied")
... resto do código
...
```

## Critérios de Aceite

- [ ] Swap requer ownership da API key
- [ ] Teste: usuário A não pode fazer swap com key de usuário B
- [ ] Adicionar rate limiting (máximo N swaps por minuto)
- [ ] Logar todas operações de swap para auditoria